

(34) Linked HashSet.

- ① Linked HashSet is a implemented class of set, cloneable, & Serializable interface (java.util)
- ② It extends HashSet class.
- ③ It will not allow to add duplicates value. it will maintain inserting order.
- ④ It is non-synchronised class.
- ⑤ ————

(35) HashMap. (JDK 1.2)

- ① HashMap is a implemented class of Map interface which is present in java.util package.
- ② HashMap extends AbstractMap class.
- ③ HashMap is non-synchronised class. (Not thread safe).
- ④ HashMap contains value based on the key.
- ⑤ It may have single null key or multiple null values.
- ⑥ HashMap doesn't maintain order.
- ⑦ HashMap is faster compare to TreeMap.
- ⑧ HashMap class use the hashtable.

(36) TreeMap.

(Map) NavigableMap

- ① TreeMap is implemented class of SortedMap interface which is present in java.util.
- ② TreeMap extends AbstractMap class.
- ③ TreeMap is non-synchronised class.
- ④ TreeMap contains value based on the key.
- ⑤ TreeMap is sorted by keys.
- ⑥ It cannot have a null key but have multiple null values.
- ⑦ keeps one in ascending order.
- ⑧ It stores the object in tree structure.
- ⑨ TreeMap is slow compare to HashMap.
- It internally use Redblack tree.

(37) LinkedHashMap

- ① LinkedHashMap is implemented class of map interface which is present in java.util.
- ② It extends HashMap class.
- ③ It is non-synchronised class.
- ④ It store the value in the 'key-pair combination'.
- ⑤ It cannot have duplicate elements
- ⑥ It may have one null key or multiple null values
- ⑦ It can be maintain insertion order.

(38) List Interface Methods ---

- ① add(e) → Appends the specified elements to the end of list
→ It will return boolean.
- ② add(int index, e) → Insert the specified element at specified position. (return void).
- ③ addAll(collection) → used to add all of the specified elements to the specified collection. (return boolean)
- ④ addAll(int index, c) → used to add all of the specified elements to the specified collection at specified position. (boolean)
- ⑤ clear() → clear method used to remove all the elements from collection. (void)
- ⑥ contains(object o) → check that object is present or not (boolean)
- ⑦ get(int index) → Return the element at the specified position in this list. (return string)
- ⑧ hashCode() → Return the hashcode value for this list. (return int)
- ⑨ indexOf(object o) → return the index of first occurrence. return (int)
- ⑩ isEmpty() → If list is empty return true. (boolean)

- ① remove (int index) → Remove the element at the specified position in this list. (Return String)
- ② remove (Object) → (Return Boolean)
- ③ removeAll (Collection) → Removes from this list all of its elements that are contained in specify collection.
(Return Boolean)
- ④ set (int index, String element) → Replace the element at specific position in this list
- ⑤ size () → return the number of element in this list
(Return int)
- ⑥ toArray () → return an array containing all the elements of list.
return (Object[])
- ⑦ iterator () → Return an iterator over the elements in this list
(Return String)
- ⑧ subList (int from index, int to index) return (List<String>)

③9 Hashtable

- ① Hashtable is a implemented class of Map interface which is present in java.util.
- ② It extends Dictionary class.
- ③ Hashtable doesn't allow null key or value
- ④ It is synchronised class.
- ⑤ Hashtable is a legacy class.

41) Methods of Set Interface

Method Name	Use	return type
<code>add(String e)</code>	Add the specified element to this set if it is already not present	<code>boolean</code>
<code>addAll(Collection)</code>	Adds all the elements in the specified collection to this set if they are not already present	<code>boolean</code>
<code>contains(Object o)</code>	Returns true if this set contains the specified element	<code>boolean</code>
<code>isEmpty()</code>	Return true if set is empty	<code>boolean</code>
<code>remove(Object o)</code>	Remove the specified element from this set.	<code>boolean</code>
<code>removeAll(Collection)</code>	Removes the from this set all of its element that are contained in specified collection	<code>boolean</code>

size()

returns the number of elements in this set

int

iterator()

Returns an iterator over the element in this set

Iterator<String>

clear()

Remove all of the elements

void

(42) Methods of MAP Interface,

containsKey(obj key)

Return true if map contain a mapping for specified key

boolean

containsValue(obj value)

Returns true if this map contains one or more keys to specified value

boolean

equals(object o)

Return true if the given object is also a map & the two maps represent the same mapping

boolean

get(object key)

Return the value to which the specified key is mapped.

String

hashCode()

returns the hashCode
value for this map

int

isEmpty()

return true if this map
contains no key-value

boolean

put(int key, String value)

Associates the specified value
with the specified key in
this map.

String

remove(Object key)

Removes the mapping key
from this map

String

replace(int key, String value)

replaces the entry for
the specified key only if
it is currently mapped
to same value

String

entrySet()

returns a set view of the
mapping contained in this
map.

Set<Entry<
Integer, String>>

keySet()

Return a set view of
keys contained in
this map

Set<Integer>

values() returns a collection view of values contained in the map

(Collection, List)

42) Exception Handling

① An unexpected events which occurs at the runtime that is during execution of a program, that disrupts the normal flow of a program is called exception. (Abnormal termination)
② defining the alternative ways to recover that exception is called exception handling. (Normal termination)

③ exceptions are occur because of programmers.

④ we can recover exceptions.

⑤ There are two types of exception

① compile time (checked exception) ② Runtime (unchecked)

⑥ Throwable is a parent class of exception & error

⑦ exception is parent class of compile time exception classes & runtime exception classes.

⑧ errors are occurs because of lack of sys. resources.

⑨ we can't recover error

⑩ Type of error - runtime error

43)

Compile time exception

① which are checked by compiler at compile time,

② program gives compilation error, if a method throws compile time exception

③ checked exception occur when chances of failure is high

④ They are direct subclass of exception class but do not inherit from runtime exception

runtime exception.

① which are not checked at compile time

② program compiles successfully because compiler not able to check that exception

③ unchecked exception occur mostly due to programmer's mistakes.

④ They are direct subclass of runtime exception class.

Compile time

Exception

↓ child class

- ex. ① ClassNotFoundException ② NoSuchMethodException
③ IOException ④ SQLException
⑤ FileNotFoundException ⑥ EOFException

Runtime exception

↓ child class

- ex ① ArithmeticException ② ClassCastException
③ NullPointerException ④ IndexOutOfBoundsException

(44) Try, catch & finally.

- ① Try block always execute

(44) Abstraction

- ① Abstraction is the process of hiding implementation details from the user, only essential functionality will be provided to the user.
- ② Example → we can see the diff. functionality options (Withdraw, check bal, change pin) provided on ATM screen but can't see the actual program developed to run these functions.
- ③ Two ways to achieve abstraction
- ① Abstract class
 - ② Interface.

(45) Abstract class & Abstract method.

- ① class which contains abstract keyword in its declaration is known as Abstract class.
- ② Abstract class may or may not contain abstract method.
- ③ we can achieve partial abstraction by using abstract class.
- ④ Method without body is called abstract method.
- ⑤ If the class has at least one abstract method then we have to compulsorily declare class as an abstract class.
- ⑥ If you inherit abstract class then we have to provide implementation to all abstract methods in child class.
- ⑦ we can not create object of abstract class.

(46) Interface

- ① Interface is a reference datatype, it is used to represent

real world entities.

Interface is fully unimplemented class.

- ② Interfaces are similar to abstract class but having all the methods of abstract type.
- ③ Interface is blueprint of the class. It specifies what a class must do & not how?

Interfaces are used to achieve abstraction.

- ⑤ By using interface we can achieve fully abstraction.

- ⑥ we can not create object of interface.

- ⑦ Interface does not contain any constructor.

- ⑧ Interface can not contain instance members.

- ⑩ By default interface contain public static final variable.

- ⑪ By default interface contain public abstract method.

④⑦ Difference Interface & Abstract class.

Interface

- ① Interface contains method that must be abstract
- ② Interface contain variables that must be static & final
- ③ member in an interface public by default
- ④ Interface is used to implement
- ⑤ Interface can be used to achieve multiple inheritance
- ⑥ Interface can extend another interface
- ⑦ Interface is completely abstract
- ⑧ Interface is more flexible because one class can extend one superclass but implement multiple interface.

Abstract class

- ① Abstract class can contain concrete method.
- ② Abstract class can contain non-final & final variable.
- ③ Abstract class may contain non-public members
- ④ abstract class used extend.
- ⑤ abstract class can be used as a single inheritance
- ⑥ abstract class can extend another class & implement multiple interface
- ⑦ abstract class can extend one superclass but implement multiple interface

(48) Encapsulation

- ① Hide the data behind a method is called encapsulation.
- ② Encapsulation is group mechanism where we can group of variables & methods into single unit is called class. class is the best example of encapsulation.
- ③ Realtime example → As a driver you know how to start the car by pressing start button & internal details of the starting operation are hidden from you. so the entire starting process is hidden from you otherwise we can tell starting operation is encapsulated from you.
- ④ The main purpose of encapsulation is data security.
- ⑤ Data hiding means making variables are private.
- ⑥ we can achieve encapsulation by making variables are private & by making methods are public.
- ⑦ we have required two methods also i.e getter & setter.

(49) Polymorphism

- ① The process of providing multiple implementation to a method to perform operation on the basis of subtype or parameter type.
- ② Two types of polymorphism.
 - ① Compiletime —
 - ② Runtime poly —
- ③ compiletime polymorphism handle by compiler
- ④ Runtime polymorphism handle by JVM.
- ⑤ we can achieve compiletime polymorphism by using method overloading
- ⑥ we can achieve runtime - it by using method overriding.

(50) Method Overloading

- ① If the class having multiple methods with the same but different argument list is called method overloading. —
- 1 or more method must be same name & they can

be define in same class.
Methods must have different argument list.

③

- ① → No. of argument can be different
- ② → sequence of argument —
- ③ → type of argument —

④ overloaded method may have same or different return type.

⑤① Method overriding.

① Redefine parent class methods in child class is called method overriding.

② If child class has same method name as in the parent class is known as method overriding.

③ child class methods must have same parameter list as in the parent class.

④ There is must IS-A relationship.

⑤ Method overriding occurs in two class that having parent-child relationship.

⑥ parent class methods & child class method return type must be same.

⑤② Constructor Overloading.

① If the class having multiple constructor with the same but diff. argument list is called constructor overloading.

② Two or more constructor must be same & they can be define in same class.

③ Constructors must have different argument list.

④ → No. of argument list can be different

→ sequence of argument —

→ type of argument —

(53) Difference Overloading & Overriding.

Overloading

- ① If the class having multiple methods with same name is call overloading.
- ② Overloading performs in same class.
- ③ Method overloading is the example of compile time polymorphism.
- ④ Overloaded method may have same or diff return type.

Overriding

- ① Redefine parent class methods in child class is called overriding.
- ② overriding occurs in two class that having parent-child relation.
- ③ Method overriding is the example of runtime polymorphism.
- ④ parent class methods & child class methods must have same return type.

(54) Difference Overriding & Hiding.

Overriding

- ① Instance Method can only override.
- ② In method overriding method resolution based on runtime object.
- ③ Runtime polymorphism.

Hiding

- we can hide static method & Global variable.
- ② In method hiding method resolution based on method type.
- ③ Compile time polymorphism.

(55)